

Neural networks estimation of the density in molecular clouds : Benchmark on Orion B dense cores

Z. Ribeiro^{1*} and P. Hily-Blant¹

Univ. Grenoble Alpes, CNRS, IPAG, 38000 Grenoble, France

Received September XX, 20XX

ABSTRACT

Context. /

Aims. This work explores the use of different neural network architectures to estimate the mass-weighted average volume density along the line of sight in molecular clouds, as well as the average density of dense cores. We aim to provide a full benchmark of these architectures, highlighting their advantages and limitations in this physical context.

Methods. We use synthetic datasets generated from magnetohydrodynamical simulations from which column and average volume density maps were derived. These datasets allows the training of different neural networks: a CNN (UNet) and two conditioned generative ones: cINNs and DDPMs. We apply different numericals and physicals metrics to test the robustness, confidence and limitations of each method.

Results. /

Conclusions. /

Key words. ISM: clouds – ISM: structures – stars: formation – machine learning – neural networks – mass function

1. Introduction

2. Methods

Mini intro

Except for making the simulations, all steps, from creating the datasets and training neural networks to applying them to observations, are performed using our code POLARIScore¹, which includes our implementations of network architectures using PyTorch (Paszke et al. 2019).

2.1. MHD Simulations and Datasets

Neural networks need to be trained on synthetic data, where they can learn the relevant correlations and underlying physics. For segmentation tasks, such training data can consist of human-generated labels on observations (see, for example, filament-identification studies(CITE)).

However, in our task of estimating a physical quantity in molecular clouds, we do not have access to true observational values. This is where simulations become essential, since they provide full access to all required physical quantities.

The properties of the simulations used to train the neural network are therefore extremely important and directly influence the network's performance.

These datasets are created using magnetohydrodynamical (MHD) simulations of a star-forming molecular cloud: ORION (Ntormousi & Hennebelle 2019), publicly available through the GALACTICA² database.

These simulations are isothermals, model MHD turbulence and

have a size of 66.1 pc with an initial resolution of 0.01 pc, reaching a few hundred AU using an AMR grid. The simulations were performed using RAMSES (Teyssier 2002), with self-gravity applied, star-formation treated using Lagrangian sink particles and ideal MHD resolved and no chemistry.

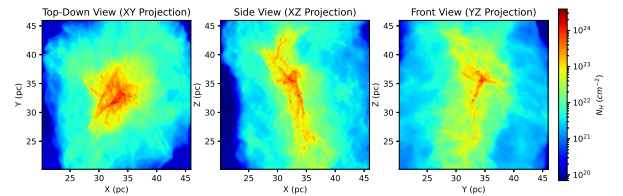


Fig. 1: Column densities for 3 axes of the same molecular cloud from the low magnetic field ORION simulation. (Ntormousi & Hennebelle 2019)

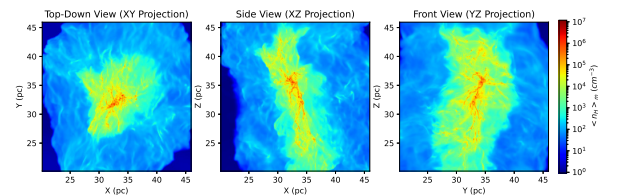


Fig. 2: Mass-weighted average volume densities for 3 axes of the same molecular cloud from the low magnetic field ORION simulation. (Ntormousi & Hennebelle 2019)

[...]

* zack.ribeiro@univ-grenoble-alpes.fr

¹ <https://github.com/ZackRibeiro/POLARIScore>

² http://www.galactica-simulations.eu/db/STAR_FORM/ORION/

2.2. Training

Training a neural network is a crucial step in achieving good performance. There are multiple choices that we need to make to obtain optimal results.

- We want the model, with N free parameters (weights), to converge to a global minimum of the loss function rather than getting stuck in a local well. Therefore, the choice of optimizer, which performs backward error propagation (adjusting the model's weights), is important. Here, we use ADAM (Kingma & Ba 2017).
- Loss function: A well-chosen loss function is essential for accurately representing the problem. Here, we generally use the mean squared error or minimizing the negative log-likelihood depending on the architecture.
- Learning rate: The learning rate is another critical factor, it determines the step size the optimizer takes in the loss function during each update (epoch). Ideally, it should decrease when the loss reaches a plateau. To manage this, we use a *scheduler*.

Each epoch, i.e., learning step, the model processes all the dataset, column density images, and predicts their mass-weighted density. A way to artificially increase the size of the training batch is to apply random transformations to the images at each epoch (Yang et al. 2023), such as random rotations and random vertical or horizontal flips (no deformations). Afterward, the prediction loss is computed using the chosen loss method, and backward error propagation is performed. Then, a new epoch begins, repeating the same steps.

2.3. Machine learning

Talk about each architectures, why/how they work, and also architecture complexity.

Why these architectures ? Why not GAN (bcs mode collapse) ?
Why not Flow matching (alternative to ddpms, no time to run it)

2.3.1. Convolutional Neural Network

2.3.2. Conditional Invertible Neural Network

2.3.3. Denoising Diffusion Probabilistic Model

2.4. Application

3. Results

3.1. Benchmark on simulation data

MSE, Spatial Error, Accuracy, Residuals; Inference time, Complexity (=Parameters number), Robustness over training set changes, Training set size needed, Training stability

3.2. Test on Orion B

Structures metrics: Density PDFs, Fractal dimension, Typical size of structures (?), ...; Dense cores metrics: Direct comparison to konyves, Direct view of cores, $N_H(n_H)$, DCMF

3.3. Summary

Table: Models vs Metrics

Article number, page 2

4. Discussions

Other methods to find mass-weighted density : Gaches & Grudić (2024), Orkisz & Kainulainen (2024).

5. Conclusions

References

- Gaches, B. A. L. & Grudić, M. Y. 2024, A Probabilistic Model to Estimate Number Densities from Column Densities in Molecular Clouds, version Number: 1
- Kingma, D. P. & Ba, J. 2017, Adam: A Method for Stochastic Optimization, arXiv:1412.6980 [cs]
- Könyves, V., André, P., Arzoumanian, D., et al. 2020, Astronomy & Astrophysics, 635, A34
- Ntormousi, E. & Hennebelle, P. 2019, Astronomy & Astrophysics, 625, A82
- Orkisz, J. H. & Kainulainen, J. 2024, On volume density and star formation in nearby molecular clouds, version Number: 1
- Paszke, A., Gross, S., Massa, F., et al. 2019, PyTorch: An Imperative Style, High-Performance Deep Learning Library, arXiv:1912.01703 [cs]
- Teyssier, R. 2002, Astronomy & Astrophysics, 385, 337
- Yang, S., Xiao, W., Zhang, M., et al. 2023, Image Data Augmentation for Deep Learning: A Survey, arXiv:2204.08610 [cs]

90

100

Appendix A: From mass-weighted averages to core densities

In this work, the neural networks aim to predict the mass-weighted average volume density:

$$\langle n_H \rangle_m(x, y) = \frac{\int_0^L n_H(s, x, y) m(s, x, y) ds}{\int_0^L m(s, x, y) ds}, \quad (\text{A.1})$$

where x and y are the plane-of-sky positions, s is the position in line of sight axis, L the cloud length, n_H the hydrogen volume density, m the hydrogen mass at (x, y, s) .

If we discretize the density along the l.o.s into M components with constant volume density in it (e.g for a datacube simulation, M corresponds to the number of cells), then Eq A.1 becomes

$$\langle n_H \rangle_m = \frac{\sum_{i=0}^M n_{H,i}^2 \times L_i \mu m_H}{\sum_{i=0}^M n_{H,i} \times L_i \mu m_H} = \frac{\sum_{i=0}^M n_{H,i}^2 L_i}{\sum_{i=0}^M n_{H,i} L_i}, \quad (\text{A.2})$$

where L_i is the length of component i . This quantity is chosen mainly because this weighted average density is close to the average density of dense structures along the l.o.s such as dense cores.

However, when a dense core is embedded by a very diffuse region (i.e outside filamentary structures), a correction is required to convert the mass-weighted average density $\langle n_H \rangle_m$ into the true average volume density of the dense core $n_{H,c}$. To simplify notation, we denote the average core density simply by n_c and volume densities without the H .

Consider two components along the line of sight ($M = 2$):

- A diffuse medium with length L_d and with constant density n_d .
- The dense core with length $L_c = 2r_c$ where r_c is the core radius, and with constant density n_c .

Equation A.2 then reduces to

$$\langle n_H \rangle_m = \frac{n_c^2 L_c + n_d^2 L_d}{n_c L_c + n_d L_d}, \quad (\text{A.3})$$

To estimate n_c , two parameters must be assumed: n_d and L_d , while L_c is derived from observations as two times the core radius. The column density: $N_H = n_c L_c + n_d L_d$ removes one degree of freedom, giving

$$n_c = N_H \left(\frac{1 + \sqrt{1 - \left(\frac{L_d}{L_c} + 1 \right) \left(1 - \frac{\langle n_H \rangle_m L_d}{N_H} \right)}}{L_c + L_d} \right), \text{ if } L_d \text{ assumed.} \quad (\text{A.4})$$

$$\text{or } n_c = \frac{n_d}{2} \left(1 + \sqrt{1 - \frac{4N_H}{L_c n_d} \left(1 - \frac{\langle n_H \rangle_m}{n_d} \right)} \right), \text{ if } n_d \text{ assumed.} \quad (\text{A.5})$$

But these corrections can lead to $\langle n_H \rangle_m$ being higher than n_c , because the conditions $n_d > 0$ or $L_d > 0$ are not enforced. To rectify this, we clamp the core density as $n_c = \max(n_c, \langle n_H \rangle_m)$, with n_c computed from one of the previous formulas.

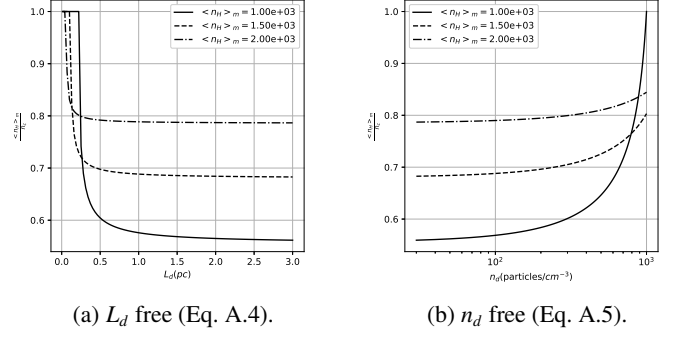


Fig. A.1: Correction $\langle n_H \rangle_m \rightarrow n_c$ for different $\langle n_H \rangle_m$; the vertical axis shows the ratio $\langle n_H \rangle_m / n_c$. The used parameters are $r_c = 0.05$ pc and $N_H = 10^{21}$ cm⁻².

If one wants to assume specific values for L_d and n_d :

$$n_c = \frac{\langle n_H \rangle_m}{2} \left(1 + \sqrt{1 + \frac{4L_d n_d}{L_c \langle n_H \rangle_m} \left(1 - \frac{n_d}{\langle n_H \rangle_m} \right)} \right) \quad (\text{A.6})$$

Figure A.1 displays the functions $n_c(L_d)$ (top) and $n_c(n_d)$ (bottom) for various values of $\langle n_H \rangle_m$. A notable feature is that n_c rapidly converges toward an upper limit when L_d becomes large or when n_d is small. The limits are

$$\lim_{L_d \rightarrow +\infty} n_c = \lim_{n_d \rightarrow 0} n_c = \sqrt{\frac{\langle n_H \rangle_m N_H}{L_c}}, \quad (\text{A.7})$$

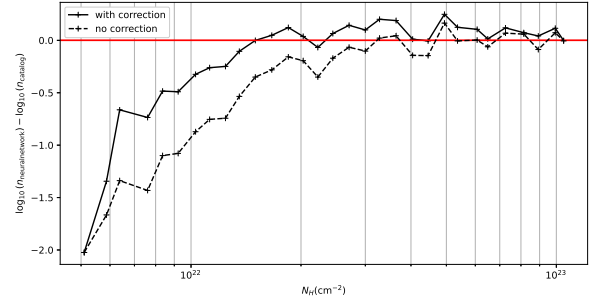


Fig. A.2: Differences in log space between the catalog core densities and the neural-network-predicted densities as a function of column density. The cloud is Orion B, neural network is UNet, and the dense-core catalog is from Könyves et al. (2020). The thick black line shows the density difference using the correction (Eq. A.4); the dotted black line shows the result without the correction. The red horizontal line indicates zero difference, i.e., where the neural-network prediction matches the catalog density.

We see in Figure A.2 that the correction indeed increases the dense-core density by an order of magnitude in diffuse regions, i.e., at low column densities. However, at high column densities, the two-medium approximation no longer holds, and therefore we can define an upper threshold, for example 3×10^{22} , above which the density correction should not be applied.

Appendix B: Validation error and bias

Residuals distributions; Correction of the bias; Residuals error propagation to metrics (MC);

150 Appendix C: U-Net architecture variations

C.1. U-Net with Kolmogorov Arnold Networks

C.2. Size and Context embedding in U-Net